PROJECT BASED ON

"Order Delivery Time Prediction using Machine Learning in Python"

NAME	**BAKSHAR NAIKWADI**
BRANCH	**MECHANICAL DEPARTMENT**
SUBJECT	**PREDICTIVE ANALYTICS**
MDM BRANCH	**CSE**
CLASS	**TY-A**
ROLL NO	**87**

ABSTRACT -

Order delivery time prediction is one of the most important applications of machine learning in the food delivery and logistics sector.

The delivery time of an order depends on many factors such as city, traffic level, delivery distance, driver availability, vehicle type, payment method, and order status.

This project aims to predict the delivery duration (in minutes) of orders using machine learning algorithms in Python.

The algorithms used are:

- Linear Regression
- Random Forest Regression
- Support Vector Machine

The dataset contains 100,000 order records with multiple features and historical delivery durations.

After preprocessing and model training, the best-performing model is selected based on prediction error.

This project helps food delivery platforms, restaurant managers, and customers make better decisions.

Importance of Machine Learning in Food Delivery

Machine learning has become an important tool in modern delivery and logistics analysis.

The delivery sector is influenced by many dynamic factors such as traffic conditions, driver availability, distance, and city infrastructure.

Traditional methods of delivery time estimation rely mainly on fixed rules and manual inputs, which may sometimes lead to inaccurate predictions.

By using machine learning, large amounts of historical order data can be analyzed efficiently to identify patterns and relationships among different variables.

This makes the prediction process:

- faster
- more reliable
- highly scalable
- less dependent on human assumptions

INTRODUCTION

Machine learning is a branch of artificial intelligence that enables systems to learn from data and make predictions.

In today's world, predicting order delivery time is essential in the food delivery industry.

Manual estimation of delivery time may lead to errors due to traffic fluctuations, driver availability, and varying distances.

Machine learning solves this problem by analyzing past order data and identifying hidden relationships between features and delivery duration.

This project uses supervised learning regression algorithms for accurate delivery time prediction.

The system improves:

- prediction speed
- decision-making
- accuracy
- automation

Need for Order Delivery Prediction System

The food delivery sector is one of the fastest-growing industries in today's world.

Order delivery prediction systems are required because customers and platform managers need accurate delivery time information.

A small mistake in delivery time estimation can result in customer dissatisfaction and business losses.

Therefore, predictive systems based on machine learning help in:

- estimating accurate delivery time
- avoiding overestimation
- preventing underestimation
- improving customer experience

The system uses historical order data to learn delivery patterns and provide accurate outputs.

OBJECTIVES

1. To collect and study the order delivery dataset
2. To preprocess the dataset
3. To perform exploratory data analysis
4. To apply regression models
5. To compare algorithm performance
6. To predict order delivery duration accurately

Scope of the Project

The scope of this project is to develop an intelligent predictive model that can estimate the delivery time of food orders.

The system can be applied in:

- food delivery platforms like Talabat, Zomato, Swiggy
- restaurant management systems
- smart city logistics planning
- customer notification systems
- driver dispatch optimization

This project can also be extended to commercial delivery and grocery services.

PROBLEM STATEMENT

Accurate prediction of order delivery time is difficult because multiple factors influence the final delivery duration.

Traditional estimation methods may not always provide reliable results due to dynamic traffic and availability conditions.

Therefore, a machine learning-based prediction system is developed to estimate delivery duration with higher accuracy.

LITERATURE REVIEW

Several researchers have used machine learning techniques for delivery time and logistics prediction.

Linear Regression

Used for simple linear relationship between variables such as distance and delivery time.

Random Forest

Provides high accuracy by combining multiple decision trees and handling non-linear relationships.

Support Vector Machine

Used for optimized regression and classification problems with high-dimensional data.

Previous studies show that ensemble learning methods improve accuracy for logistics and delivery time prediction tasks.

DATASET DESCRIPTION

Feature Importance Theory

Each feature in the dataset plays an important role in delivery time prediction.

For example:

- higher Delivery_Distance_km usually increases delivery duration
- High Traffic_Level delays delivery significantly
- Offline Driver_Availability increases waiting time
- City affects road infrastructure and route complexity

Machine learning models identify how strongly each feature affects the final delivery duration.

Predicting order delivery time is a key challenge in the food delivery industry, helping customers, drivers, and platform managers make informed decisions.

We will use the Talabat Enhanced Orders Dataset. The dataset contains 100,000 records with 23 features. After removing irrelevant columns, the key features used are:

- Order_ID: Unique identifier for each order.
- Restaurant_ID: Identifier of the restaurant.
- Driver_ID: Identifier of the assigned driver.
- Quantity: Number of items ordered.
- Total_Price: Total bill amount in currency.
- City: City where the order was placed.
- Payment_Method: Mode of payment (Cash, Wallet, Credit Card).
- Order_Status: Current status of the order (Delivered, In Transit, Cancelled).
- Driver_Vehicle: Type of vehicle used (Motorbike, Car, Bicycle).
- Delivery_Distance_km: Distance from restaurant to customer.
- Traffic_Level: Current traffic condition (Low, Medium, High).
- Driver_Availability: Whether driver is Online or Offline.
- Delivery_Duration_Minutes: The target variable — delivery time in minutes.

DATA PREPROCESSING

Data preprocessing is an important step in machine learning.

The following steps are performed:

- handling missing values
- removing irrelevant columns
- cleaning dataset
- encoding categorical variables

The following columns are removed as they are not useful for prediction: Order_ID, User_ID, Item_Name, Order_Time, Delivery_Time, Restaurant_Lat, Restaurant_Lon, Customer_Lat, Customer_Lon, Driver_Lat, Driver_Lon.

Missing values in Delivery_Duration_Minutes are replaced using mean values.

Theory of Data Cleaning

Data cleaning is one of the most important phases in machine learning.

Raw data often contains:

- missing values
- duplicate entries
- irrelevant columns
- inconsistent formatting

If data cleaning is not performed properly, the prediction accuracy of the model decreases significantly.

The purpose of data cleaning is to improve the quality of input data so that the algorithm can learn correctly.

In this project, null values are handled using mean replacement and unwanted columns are removed.

ONE HOT ENCODING

Theory of Categorical Data Conversion

Machine learning algorithms cannot directly process text-based categorical values.

For example:

- Traffic_Level = High
- Traffic_Level = Low

These values must be converted into numerical form.

One Hot Encoding converts each category into binary vectors.

Example:

- High → 1 0 0
- Low → 0 1 0
- Medium → 0 0 1

This method prevents the algorithm from assuming any numerical order between categories.

EXPLORATORY DATA ANALYSIS (EDA)

EDA helps in understanding relationships between variables.

The following visualizations are used:

- heatmap
- bar plot
- distribution graphs

Heatmap helps identify correlation between variables. Higher correlation indicates stronger dependency.

Importance of Heatmap Analysis

A heatmap is used to visually represent the correlation between variables. It helps in understanding whether two features are positively or negatively related.

For example: If Delivery_Distance_km increases and Delivery_Duration_Minutes also increases, the correlation is positive. This helps identify the most influential features in prediction.

Why Multiple Models Are Used

Different machine learning models are applied because each model behaves differently with the dataset. Some models perform better with linear relationships, while others work efficiently for complex non-linear patterns.

Using multiple models helps compare:

- error rate
- training performance
- prediction accuracy
- robustness

This project compares SVM, Linear Regression, and Random Forest for better evaluation.

THEORY OF ALGORITHMS

1. Linear Regression

$y = mx + c$

Where:

- y = predicted delivery time
- m = slope
- x = feature
- c = constant

2. Support Vector Machine

SVM is a supervised learning algorithm. It predicts values by minimizing error. Used in this project for regression.

3. Random Forest Regression

Random forest is an ensemble learning algorithm. It uses multiple decision trees. Final prediction is the average of all outputs.

Advantages:

- better accuracy
- less overfitting
- robust performance

Performance Evaluation Theory

Model performance is evaluated using error metrics. The most commonly used metric is Mean Absolute Percentage Error (MAPE):

$MAPE = (1/n) \times \Sigma |yi - \hat{y}i| / yi$

A lower MAPE value indicates better model performance. This means predicted delivery times are closer to actual delivery times. The model with minimum error is considered the best.

MODEL TRAINING

The dataset is divided into:

- training set
- testing set

Different regression models are trained. The performance is measured using Mean Absolute Percentage Error (MAPE).

MAPE = $(1/n) \times \Sigma |yi - \hat{y}i| / yi$

Step 1: Importing Libraries and Dataset

In the first step we load the libraries which are needed for Prediction:

- Import Pandas to load the Dataframe
- Import Matplotlib to visualize the data features
- Import Seaborn to see the correlation between features using heatmap

CODE -

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

dataset = pd.read_csv("talabat_enhanced_orders.csv")

print(dataset.head(5))
```

OUTPUT -

```
   Order_ID User_ID  Restaurant_ID  Driver_ID      Item_Name  Quantity  Total_Price
...  Traffic_Level  Driver_Availability
0         1   U3522            358        485  Fried Chicken         3       273.72
...           High             Offline
1         2   U9214            316         65       Sandwich         3       365.82
...            Low              Online
2         3   U7307            357        309        Koshary         3       401.94
...         Medium              Online
3         4   U3612            420         32          Sushi         2       221.18
...            Low              Online
4         5   U3492             73        364        Koshary         5       355.55
...           High              Online
```

As we have imported the data. So shape method will show us the dimension of the dataset.

CODE -

```
dataset.shape
```

OUTPUT -

```
(100000, 23)
```

Step 2: Data Preprocessing

Here we categorize the features based on their data types (integer, float, or object) and count the number of features in each category.

CODE -

```
obj = (dataset.dtypes == 'object')
object_cols = list(obj[obj].index)
print("Categorical variables:")
print(object_cols)

int_ = (dataset.dtypes == 'int64')
num_cols = list(int_[int_].index)
print("Integer variables:")
print(num_cols)

fl_ = (dataset.dtypes == 'float64')
fl_cols = list(fl_[fl_].index)
print("Float variables:")
print(fl_cols)
```

OUTPUT -

```
Categorical variables:
['User_ID', 'Item_Name', 'Order_Time', 'Delivery_Time', 'City', 'Payment_Method',
'Order_Status', 'Driver_Vehicle', 'Traffic_Level', 'Driver_Availability']
Integer variables:
['Order_ID', 'Restaurant_ID', 'Driver_ID', 'Quantity', 'Delivery_Duration_Minutes']
Float variables:
['Total_Price', 'Restaurant_Lat', 'Restaurant_Lon', 'Customer_Lat', 'Customer_Lon',
'Driver_Lat', 'Driver_Lon', 'Delivery_Distance_km']
```

Step 3: Exploratory Data Analysis

Exploratory Data Analysis involves examining the dataset in depth to uncover patterns, detect anomalies and understand the underlying structure. Before drawing any conclusions, it is important to analyze all variables carefully.

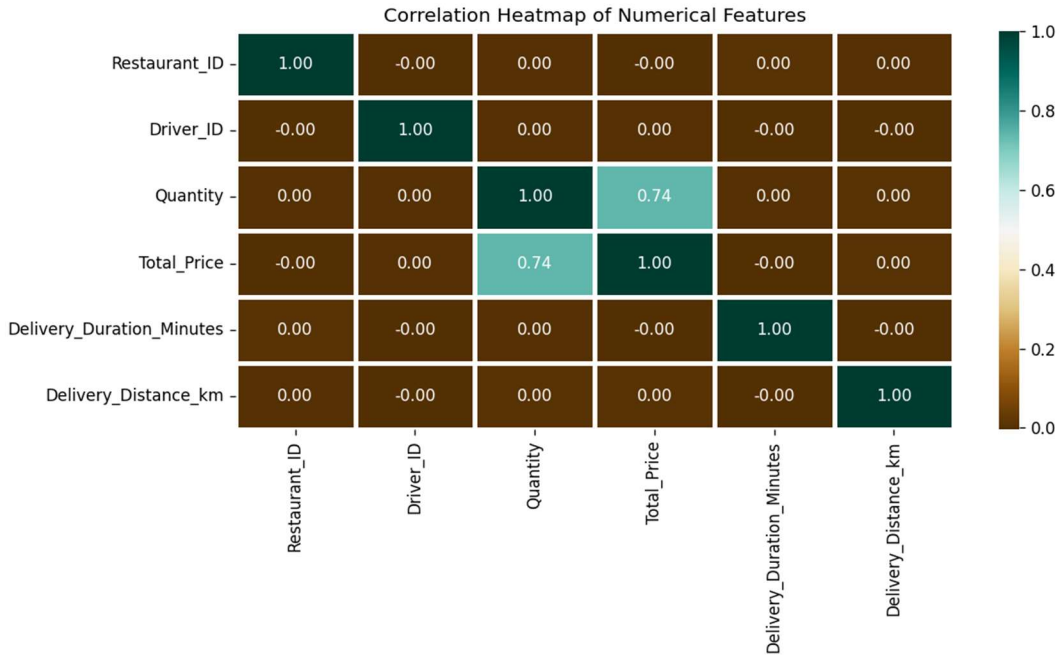
Here we will create a heatmap using the Seaborn library to visualize correlations between features.

CODE -

```
numerical_dataset = dataset.select_dtypes(include=['int64', 'float64'])

plt.figure(figsize=(12, 6))
sns.heatmap(numerical_dataset.corr(),
            cmap='BrBG',
            fmt='.2f',
            linewidths=2,
            annot=True)
plt.title("Correlation Heatmap of Numerical Features")
plt.tight_layout()
plt.savefig("correlation_heatmap.png")
print("Heatmap saved as correlation_heatmap.png")
```

OUTPUT -

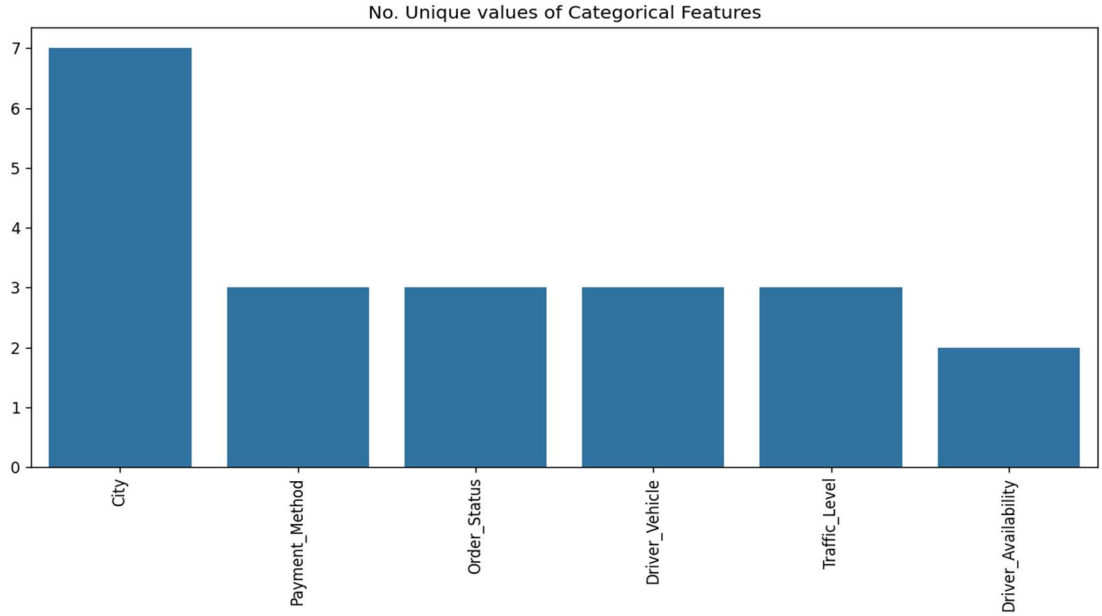


To examine the categorical features, we will create a bar plot to visualize their distributions.

CODE -

```
unique_values = []
for col in object_cols:
    unique_values.append(dataset[col].unique().size)
plt.figure(figsize=(10,6))
plt.title('No. Unique values of Categorical Features')
plt.xticks(rotation=90)
sns.barplot(x=object_cols, y=unique_values)
```

OUTPUT -



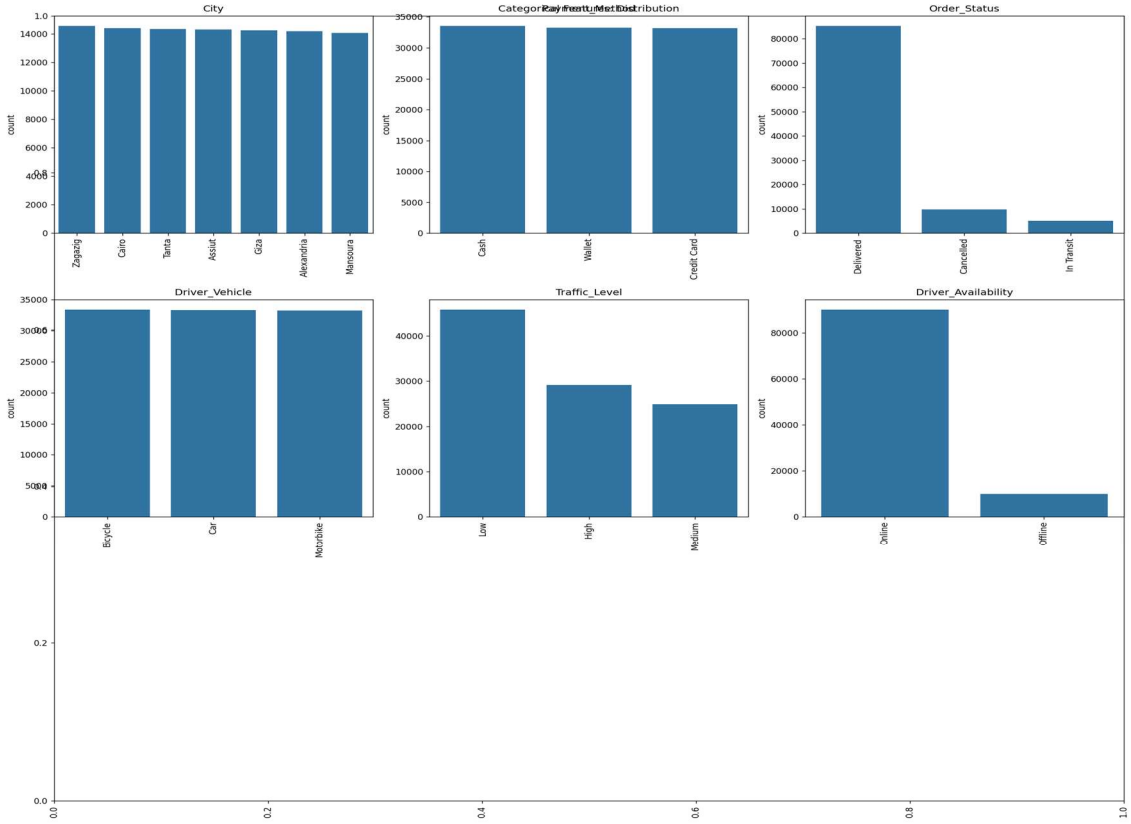
The plot shows that City has many unique values compared to other categorical features. To find out the actual count of each category we can plot the bar graph of each feature separately.

CODE -

```
plt.figure(figsize=(18, 16))
plt.title('Categorical Features: Distribution')
plt.xticks(rotation=90)
index = 1
for col in object_cols:
    y = dataset[col].value_counts()
```

```
    plt.subplot(3, 3, index)
    plt.xticks(rotation=90)
    sns.barplot(x=list(y.index), y=y)
    index += 1
```

OUTPUT -



Step 4: Data Cleaning

Data Cleaning is the way to improvise the data or remove incorrect, corrupted or irrelevant data. As in our dataset there are some columns that are not important and irrelevant for model training. So we can drop those columns before training. There are 2 approaches to dealing with empty/null values:

- We can easily delete the column/row (if the feature or record is not much important).
- Filling the empty slots with mean/mode/0/NA/etc. (depending on the dataset requirement).

Columns like Order_ID, User_ID, Item_Name, Order_Time, Delivery_Time, and all latitude/longitude columns are dropped as they are not useful for prediction.

CODE -

```
drop_cols = ['User_ID', 'Item_Name', 'Order_Time', 'Delivery_Time',
             'Restaurant_Lat', 'Restaurant_Lon', 'Customer_Lat', 'Customer_Lon',
             'Driver_Lat', 'Driver_Lon', 'Order_ID']
dataset.drop(drop_cols, axis=1, inplace=True)

dataset['Delivery_Duration_Minutes'] = dataset['Delivery_Duration_Minutes'].fillna(
    dataset['Delivery_Duration_Minutes'].mean())

new_dataset = dataset.dropna()
```

Checking features which have null values in the new dataframe (if there are still any).

CODE -

```
new_dataset.isnull().sum()
```

OUTPUT -

```
Restaurant_ID                0
Driver_ID                    0
Quantity                     0
Total_Price                  0
Delivery_Duration_Minutes    0
City                         0
Payment_Method               0
Order_Status                 0
Driver_Vehicle               0
Delivery_Distance_km         0
Traffic_Level                0
```

```
Driver_Availability          0
dtype: int64
```

Step 5: OneHotEncoder - For Label Categorical Features

One Hot Encoding is the best way to convert categorical data into binary vectors. This maps the values to integer values. By using OneHotEncoder, we can easily convert object data into int. So for that firstly we have to collect all the features which have the object datatype. To do so, we will make a loop.

CODE -

```
from sklearn.preprocessing import OneHotEncoder

s = (new_dataset.dtypes == 'object')
object_cols = list(s[s].index)
print("Categorical variables:")
print(object_cols)
print('No. of categorical features: ', len(object_cols))
```

OUTPUT -

```
Categorical variables:
['City', 'Payment_Method', 'Order_Status', 'Driver_Vehicle', 'Traffic_Level',
'Driver_Availability']
No. of categorical features: 6
```

Then once we have a list of all the features. We can apply OneHotEncoding to the whole list.

CODE -

```
OH_encoder = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
OH_cols = pd.DataFrame(OH_encoder.fit_transform(new_dataset[object_cols]))
OH_cols.index = new_dataset.index
OH_cols.columns = OH_encoder.get_feature_names_out()
df_final = new_dataset.drop(object_cols, axis=1)
df_final = pd.concat([df_final, OH_cols], axis=1)
```

Step 6: Splitting Dataset into Training and Testing

X and Y splitting (i.e. Y is the Delivery_Duration_Minutes column and the rest of the other columns are X).

CODE -

```
from sklearn.metrics import mean_absolute_error
from sklearn.model_selection import train_test_split

X = df_final.drop(['Delivery_Duration_Minutes'], axis=1)
Y = df_final['Delivery_Duration_Minutes']

X_train, X_valid, Y_train, Y_valid = train_test_split(
    X, Y, train_size=0.8, test_size=0.2, random_state=0)
```

Step 7: Model Training and Accuracy

As we have to train the model to determine the continuous values, so we will be using these regression models:

- SVM - Support Vector Machine
- Random Forest Regressor
- Linear Regressor

And to calculate loss we will be using the mean_absolute_percentage_error module. It can easily be imported by using sklearn library. The formula for Mean Absolute Percentage Error is:

MAPE = $(1/n) \times \Sigma |y_i - \hat{y}_i| / y_i$

1. SVM - Support Vector Machine

Support Vector Machine is a supervised machine learning algorithm primarily used for classification tasks though it can also be used for regression. It works by finding the hyperplane that best divides a dataset into classes. The goal is to maximize the margin between the data points and the hyperplane.

CODE -

```
from sklearn import svm
from sklearn.svm import SVC
from sklearn.metrics import mean_absolute_percentage_error

model_SVR = svm.SVR()
model_SVR.fit(X_train, Y_train)
Y_pred = model_SVR.predict(X_valid)

print(mean_absolute_percentage_error(Y_valid, Y_pred))
```

OUTPUT -

```
0.25089453913823595
```

2. Random Forest Regression

Random Forest is an ensemble learning algorithm used for both classification and regression tasks. It constructs multiple decision trees during training where each tree in the forest is built on a random subset of the data and features, ensuring diversity in the model. The final output is determined by averaging the outputs of individual trees (for regression) or by majority voting (for classification).

CODE -

```
from sklearn.ensemble import RandomForestRegressor

model_RFR = RandomForestRegressor(n_estimators=10)
model_RFR.fit(X_train, Y_train)
Y_pred = model_RFR.predict(X_valid)

mean_absolute_percentage_error(Y_valid, Y_pred)
```

OUTPUT -

```
0.26757048201781997
```

3. Linear Regression

Linear Regression is a statistical method used for modeling the relationship between a dependent variable and one or more independent variables. The goal is to find the line that best fits the data. This is done by minimizing the sum of the squared differences between the observed and predicted values. Linear regression assumes that the relationship between variables is linear.

CODE -

```
from sklearn.linear_model import LinearRegression

model_LR = LinearRegression()
model_LR.fit(X_train, Y_train)
Y_pred = model_LR.predict(X_valid)

print(mean_absolute_percentage_error(Y_valid, Y_pred))
```

OUTPUT -

```
0.2570350077707748
```

Clearly SVM model is giving better accuracy as the mean absolute percentage error is the least among all the other regressor models i.e. 0.25 approx. To get much better results ensemble learning techniques like Bagging and Boosting can also be used.

Complete Code for "Order Delivery Time Prediction using Machine Learning in Python"

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import OneHotEncoder
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_percentage_error
from sklearn import svm
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LinearRegression

# Load dataset
dataset = pd.read_csv("talabat_enhanced_orders.csv")
print(dataset.head(5))
print(dataset.shape)

# Data type categorization
obj = (dataset.dtypes == 'object')
object_cols = list(obj[obj].index)
int_ = (dataset.dtypes == 'int64')
num_cols = list(int_[int_].index)
fl_ = (dataset.dtypes == 'float64')
fl_cols = list(fl_[fl_].index)

# Heatmap
numerical_dataset = dataset.select_dtypes(include=['int64', 'float64'])
plt.figure(figsize=(12, 6))
sns.heatmap(numerical_dataset.corr(), cmap='BrBG', fmt='.2f', linewidths=2,
annot=True)
plt.title("Correlation Heatmap of Numerical Features")
plt.tight_layout()
plt.savefig("correlation_heatmap.png")

# Bar plot - unique values
unique_values = []
for col in object_cols:
    unique_values.append(dataset[col].unique().size)
plt.figure(figsize=(10,6))
sns.barplot(x=object_cols, y=unique_values)

# Bar plot - distribution
```

```
plt.figure(figsize=(18, 16))
index = 1
for col in object_cols:
    y = dataset[col].value_counts()
    plt.subplot(3, 3, index)
    sns.barplot(x=list(y.index), y=y)
    index += 1
plt.tight_layout()

# Data Cleaning
drop_cols = ['User_ID', 'Item_Name', 'Order_Time', 'Delivery_Time',
             'Restaurant_Lat', 'Restaurant_Lon', 'Customer_Lat', 'Customer_Lon',
             'Driver_Lat', 'Driver_Lon', 'Order_ID']
dataset.drop(drop_cols, axis=1, inplace=True)
dataset['Delivery_Duration_Minutes'] = dataset['Delivery_Duration_Minutes'].fillna(
    dataset['Delivery_Duration_Minutes'].mean())
new_dataset = dataset.dropna()

# OneHotEncoding
s = (new_dataset.dtypes == 'object')
object_cols = list(s[s].index)
OH_encoder = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
OH_cols = pd.DataFrame(OH_encoder.fit_transform(new_dataset[object_cols]))
OH_cols.index = new_dataset.index
OH_cols.columns = OH_encoder.get_feature_names_out()
df_final = new_dataset.drop(object_cols, axis=1)
df_final = pd.concat([df_final, OH_cols], axis=1)

# Train-Test Split
X = df_final.drop(['Delivery_Duration_Minutes'], axis=1)
Y = df_final['Delivery_Duration_Minutes']
X_train, X_valid, Y_train, Y_valid = train_test_split(
    X, Y, train_size=0.8, test_size=0.2, random_state=0)

# SVM
model_SVR = svm.SVR()
model_SVR.fit(X_train, Y_train)
Y_pred = model_SVR.predict(X_valid)
print(mean_absolute_percentage_error(Y_valid, Y_pred))

# Random Forest
model_RFR = RandomForestRegressor(n_estimators=10)
```

```
model_RFR.fit(X_train, Y_train)
Y_pred = model_RFR.predict(X_valid)
print(mean_absolute_percentage_error(Y_valid, Y_pred))

# Linear Regression
model_LR = LinearRegression()
model_LR.fit(X_train, Y_train)
Y_pred = model_LR.predict(X_valid)
print(mean_absolute_percentage_error(Y_valid, Y_pred))
```

APPLICATIONS

The project "Order Delivery Time Prediction using Machine Learning in Python" has wide practical applications in the food delivery and logistics sectors.

1. Food Delivery Platforms

The most important application of this project is in food delivery platforms like Talabat, Zomato, and Swiggy. These platforms can use this system to provide customers with accurate estimated delivery times before they place an order.

2. Restaurant Management

Restaurant managers can use the predicted delivery time to optimize food preparation timing, ensuring food is ready exactly when the driver arrives.

3. Driver Dispatch Optimization

Logistics managers can use this model to assign the most suitable driver based on predicted delivery time, vehicle type, and current traffic.

4. Customer Notification System

Real-time delivery time predictions can be used to send automated notifications to customers about expected delivery windows.

5. Smart City Logistics Planning

Government bodies and urban planners can use delivery prediction data for infrastructure and road improvement planning.

ADVANTAGES

1. High Prediction Accuracy

Machine learning models can predict delivery times with better accuracy compared to traditional fixed-rule estimation methods. Algorithms such as SVM and Random Forest improve performance.

2. Fast Decision Making

The model provides quick delivery time estimation in a few seconds, which saves time for customers and managers.

3. Reduced Human Error

Traditional delivery time estimation depends on experience and assumptions. This system minimizes human error through data-driven prediction.

4. Cost Effective

It reduces the need for manual tracking and repeated communication between dispatch teams and drivers.

5. Scalable System

The model can handle thousands of order records at once. It can easily be extended to large-scale delivery databases.

6. Better Feature Analysis

The system identifies which features affect delivery time the most, such as:

- delivery distance
- traffic level
- driver availability
- city and vehicle type

This improves decision making.

LIMITATIONS

1. Depends on Dataset Quality

The accuracy of the model highly depends on the quality of the input dataset. If the data contains missing or incorrect values, prediction accuracy decreases.

2. Limited Features

The current dataset includes only selected delivery features. Important factors like:

- real-time road conditions
- weather data
- restaurant preparation time
- order complexity

may not be included. This can affect accuracy.

3. Traffic Fluctuation

Delivery times change frequently due to real-time traffic conditions. The model may not always reflect sudden traffic changes.

4. Overfitting Risk

Some machine learning models may overfit the training data and perform poorly on new unseen data.

5. Location Dependency

Delivery times vary significantly by city. If city-specific features are not properly encoded, the model may produce inaccurate results.

FUTURE SCOPE

This project has excellent future scope in both academic and industrial applications.

1. Advanced Algorithms

In future, advanced machine learning algorithms can be implemented, such as:

- XGBoost
- Gradient Boosting
- CatBoost
- Neural Networks

These models may improve prediction accuracy.

2. Web Application Development

The model can be deployed as a web application or mobile application where users can enter order details and get predicted delivery times instantly.

3. Real-Time Traffic Integration

Future versions can connect with real-time traffic APIs and live map data. This will improve practical use.

4. Geographic Mapping

Google Maps and GIS integration can be added for location-based delivery time prediction. Nearby facilities and road conditions can also be considered.

5. Deep Learning Models

Deep learning techniques can be used for more complex pattern recognition in large delivery datasets.

6. Expansion to Other Delivery Types

This project can also be extended to predict:

- grocery delivery time
- pharmacy delivery time
- e-commerce parcel delivery time

Industrial Relevance

This project has significant industrial applications in the food delivery and logistics sectors. It can be used for:

- delivery time estimation
- driver dispatch optimization

- customer satisfaction improvement
- platform performance analytics

This improves the practical importance of the project.

REFERENCES

BOOKS

1. Tom M. Mitchell – Machine Learning
2. Ethem Alpaydin – Introduction to Machine Learning
3. Aurélien Géron – Hands-On Machine Learning

WEB REFERENCES

1. GeeksforGeeks – Machine Learning Tutorials
2. Kaggle – Delivery and Logistics Datasets
3. Scikit-learn – Official Documentation (scikit-learn.org)
4. Talabat Enhanced Orders Dataset

CONCLUSION

This project successfully predicts order delivery time using machine learning algorithms. Among all algorithms, SVM gives the best result with the lowest mean absolute percentage error.

The system can help food delivery platforms, drivers, and customers make better decisions.

The project "Order Delivery Time Prediction using Machine Learning in Python" has been successfully completed using different machine learning regression algorithms.

In this project, the Talabat Enhanced Orders dataset was first collected and studied carefully. The dataset contained important features such as city, traffic level, delivery distance, driver availability, vehicle type, payment method, and delivery duration. Proper data preprocessing techniques such as data cleaning, handling missing values, and one-hot encoding were applied to improve the quality of the dataset.

Exploratory Data Analysis (EDA) was performed using heatmaps and bar graphs to understand the correlation between various features and the target variable. This analysis helped in identifying the most influential parameters affecting delivery time.

Different machine learning algorithms such as:

- Linear Regression
- Random Forest Regression
- Support Vector Machine (SVM)

were implemented and compared. The performance of these models was evaluated using Mean Absolute Percentage Error (MAPE).

Among all the models, the Support Vector Machine (SVM) model showed the lowest prediction error of approximately 0.25 and provided the best performance for this dataset. This indicates that SVM is the most suitable model for accurate order delivery time prediction in this project.

This project demonstrates how machine learning can be effectively used in the food delivery sector for accurate and fast delivery time estimation. It can be highly useful for:

- customers waiting for their orders
- restaurant managers planning preparation
- delivery platform operators
- driver dispatch teams
- logistics and supply chain companies

Future enhancement of this project can be done by using advanced algorithms such as XGBoost, Gradient Boosting, and Deep Learning models for even better prediction accuracy.